

Hardware-in-the-Loop Simulation for LIDSS Raft Control Validation

Henry Smith (intern), Brandon Bao (mentor), Paul Nyholm (supervisor), Jesse Wynn (assistant supervisor),

The LLNL Independent Diagnostic Scoring System (LIDSS) group designs, manufactures, and deploys autonomous rafts and drones that collect terminal data during weapons testing. Testing and tuning raft control algorithms often requires taking a raft to a body of water, which can be time and resource intensive. To provide LIDSS with another tool to develop and validate improvements to their control system, I implemented a hardware-in-the-loop (HITL) simulation interface between the raft's control board (CORE board) and the commercial off-the-shelf (COTS) boat simulation software ShipSim3. I also configured the hydrodynamics settings in ShipSim3 to model the raft in simulation.



Figure 1: LIDSS raft about to be deployed on a mission. The raft shown is of the same design as the one I modeled in ShipSim3.

PROJECT DESCRIPTION

This project consists of two parts:

Interface between CORE Board and ShipSim3

Develop a high-performance interface between the raft's control board and the simulation software ShipSim3. This interface needs to respond to commands in real-time, with a maximum latency requirement of 50 milliseconds (ms) per message. The interface would also have to be simple to use, so that it would provide an effective tool to LIDSS software developers.

Hydrodynamics Configuration

Configure ShipSim3 settings so that the simulated raft reproduces the physical raft's behavior. This includes identifying hydrodynamic damping forces on the raft and providing those forces to the simulation software in equation form.

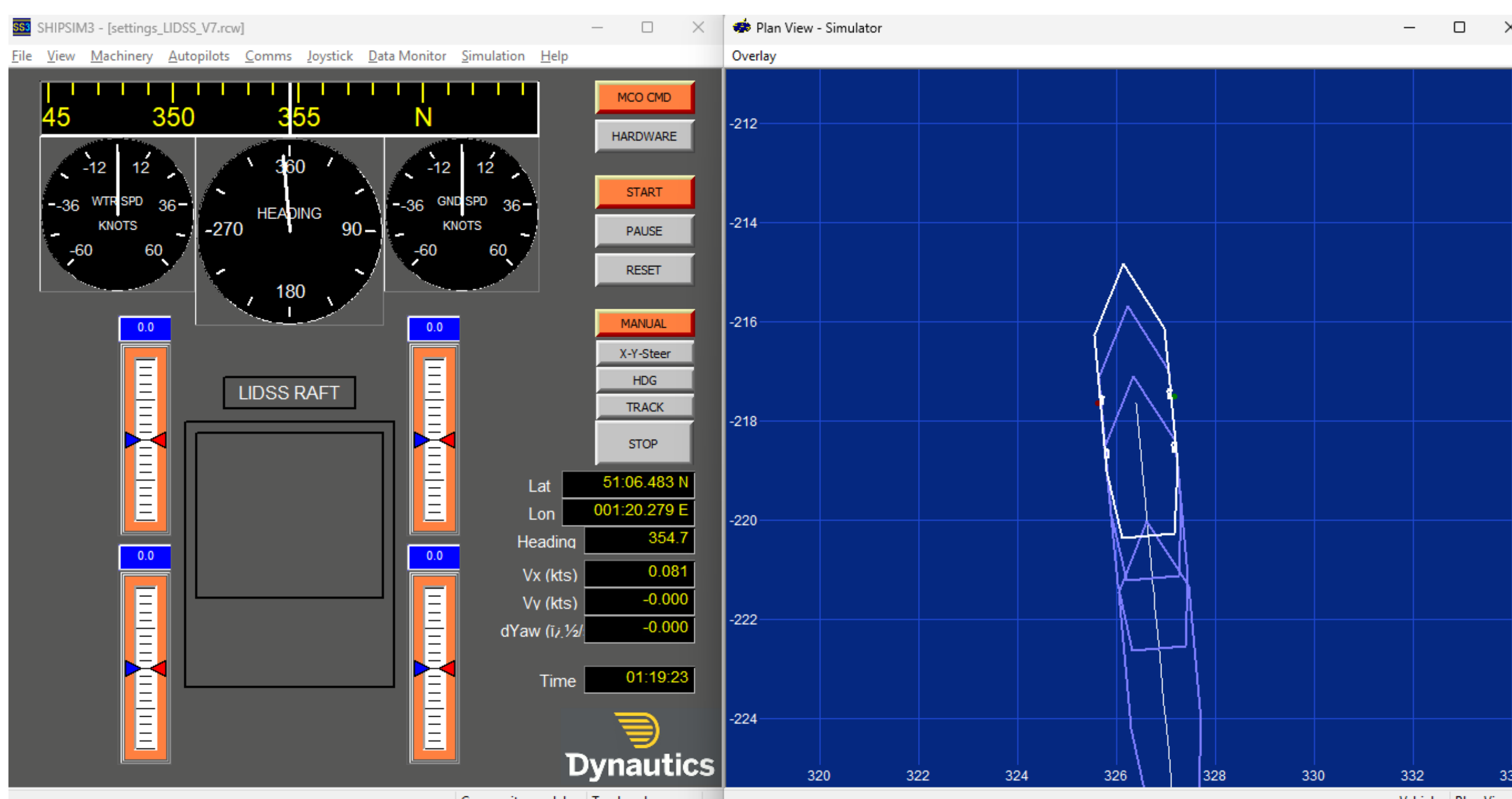


Figure 2: Visual of ShipSim3 user interface. The left panel includes readouts of GPS, heading, and velocity data, as well as control buttons. The right panel provides a visual representation of the movement of the raft.

METHODS

Communication through Python Script

To facilitate communication between the CORE board and ShipSim3, I wrote a Python script which utilizes the socket and pyserial libraries to orchestrate communication over UDP and serial. I also used a state class and the threading library to ensure that sending and receiving data happen simultaneously. Additionally, the script handles data formatting, ensuring that the data is in the correct format to be received by the CORE board and ShipSim3.

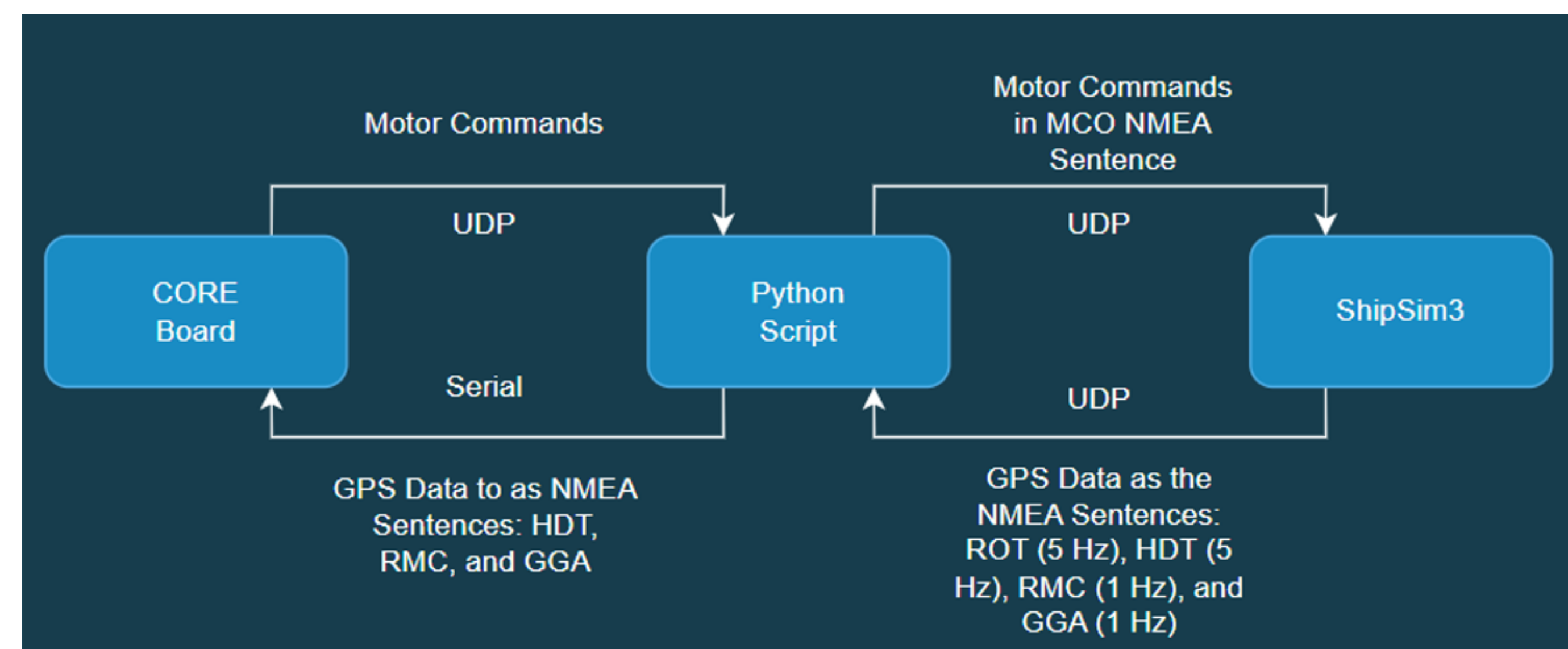


Figure 3: Graphical representation of Python script. As depicted, motor commands are sent from the CORE board to ShipSim3, while GPS data is sent from ShipSim3 to the CORE board. All data is sent over UDP connection, except for GPS to the CORE board, which is sent over serial.

Fit Data Using Linear Least-Squares Regression

To create the necessary hydrodynamics equations, I was provided with data collected from a LIDSS raft during local wet testing. This data included GPS position and heading, as well as motor commands of the raft over time. To prepare the data for fitting, I smoothed it using a Savitz-Golay filter. Then, starting with the three-degree-of-freedom equations of motion for a marine craft, I solved for the residual damping forces on the raft. Next, I used the linear least-squares method to fit the wet test data to the damping equations. This produced a hydrodynamics model of the raft that represents the raft's motion through water.

$$X = -X_u u - X_{|u|u} |u| |u|$$

$$Y = -Y_v v - Y_r r - Y_{|v|v} |v| |v| - Y_{|r|r} |r| |r|$$

$$N = -N_r r - N_v v - N_{|r|r} |r| |r| - N_{|v|v} |v| |v| - N_{ur} ur - N_{uv} uv$$

Figure 4: Format of the three damping equations entered into ShipSim3. The variables u , v , and r represent surge, sway, and yaw velocities, respectively. X , Y , and N represent the damping forces on the raft in the surge, sway, and yaw directions. The X , Y , and N terms with subscripts are the coefficients found using linear least squares.

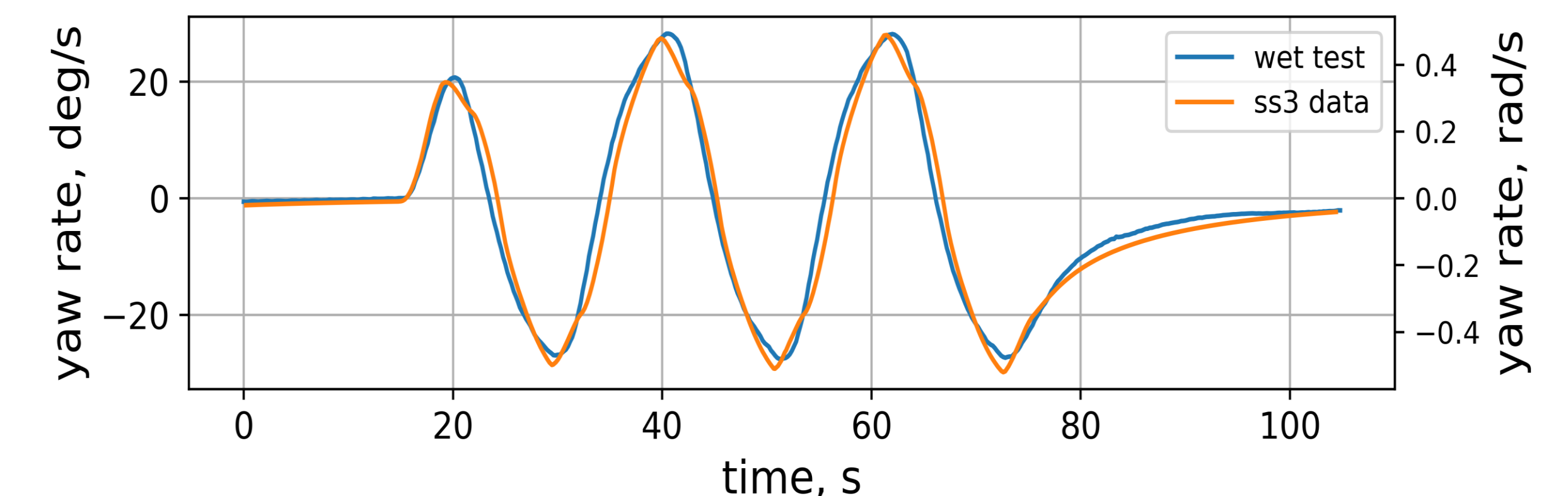


Figure 5: Graph comparing yaw rate performance of model against wet test data

RESULTS

Responsive and Efficient Communications

The Python script sends GPS data from ShipSim3 to the CORE board, and motor commands from the CORE board to ShipSim3. Also translates the data into the form that ShipSim3 and the CORE board require. UDP message forwarding completes in less than 1 ms per message, while serial message latency had a 95th percentile value of 21 ms. This meets the 20 Hz performance requirement, which permits up to 50 ms per message.

Performance of the Hydrodynamics Model

Raft configuration in ShipSim3 closely predicts real raft motion. The NRMSE (normalized root mean square error) was 15% for yaw rate and 23% for surge velocity (yaw rate was prioritized) when compared against wet test data. The model was also validated against data recorded from a real LIDSS mission, with a NRMSE of 22% for the yaw rate.

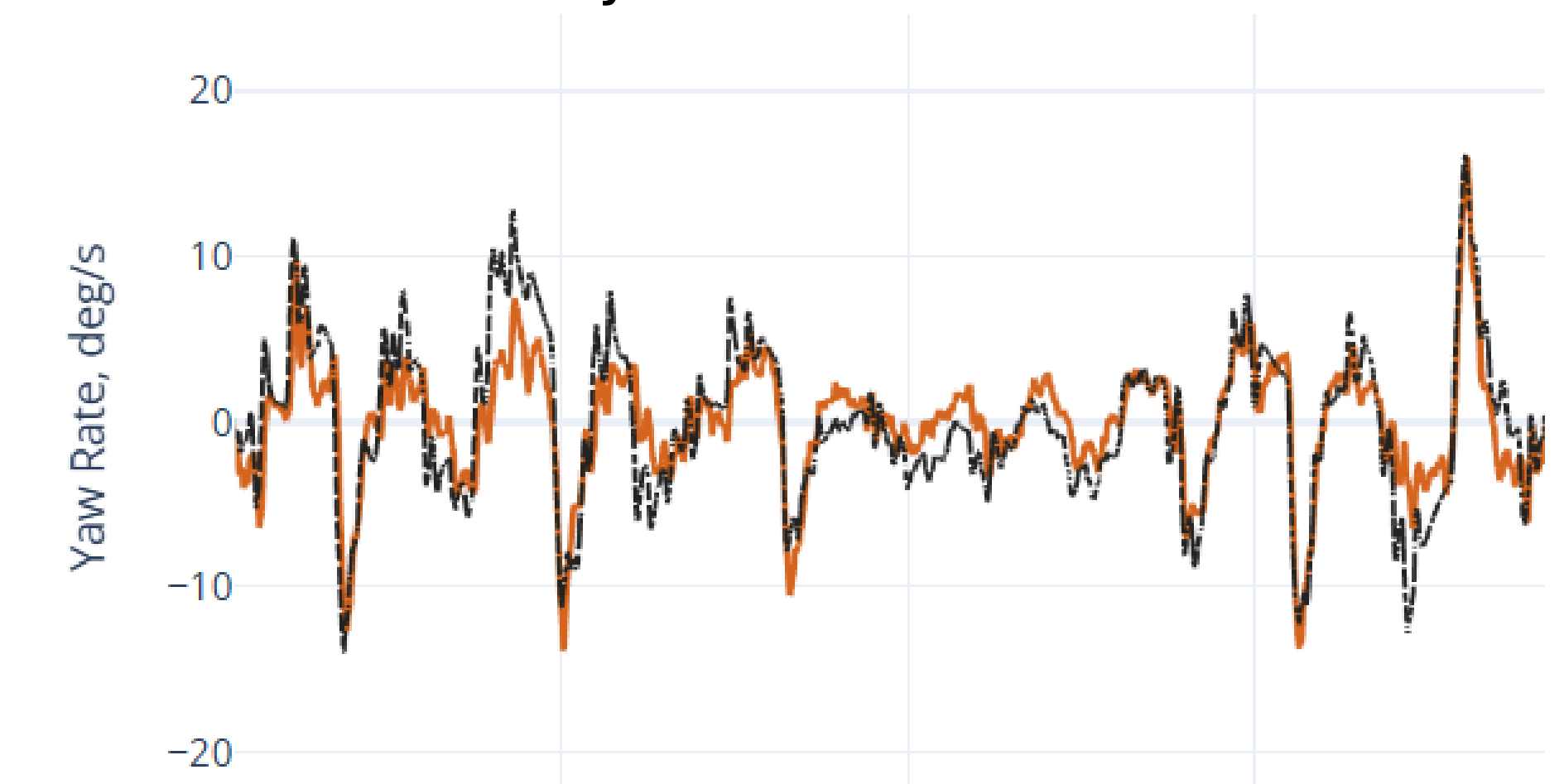


Figure 6: Graph of yaw rate of raft from mission data compared against yaw rate from the ShipSim3 model. Black denotes ShipSim3 data, and orange denotes mission data. As seen, the model represents data recorded during a real mission.

CONCLUSIONS

This tool will provide an efficient way for LIDSS software developers to validate and refine the control algorithms used on the LIDSS rafts. Additionally, this simulation will allow for LIDSS personnel to test raft behavior under various simulated circumstances, recreating mission conditions without the need for resource-intensive wet testing.

Contact

Contact Henry Smith at smith600@llnl.gov or hsmith40@byu.edu